

Microsoft SQL Server (MSSQL) Interview Guide — 3 Years Experience

Table of Contents

1. Introduction to MSSQL
2. SQL Server Architecture
3. SQL Server Editions
4. Databases and Tables
5. Data Types
6. Constraints
7. CRUD Operations
8. SQL Clauses
9. Joins
10. Set Operators
11. Functions
12. String Functions
13. Date Functions
14. Aggregate Functions
15. Group By and Having
16. Subqueries
17. Common Table Expressions (CTE)
18. Temporary Tables
19. Table Variables
20. Views
21. Indexes
22. Stored Procedures
23. User Defined Functions (UDF)
24. Triggers
25. Transactions
26. ACID Properties
27. Locks and Blocking
28. Isolation Levels
29. Deadlocks
30. Execution Plans
31. Query Optimization
32. Normalization
33. Denormalization
34. Keys in SQL Server
35. Identity Column
36. Sequences
37. Pagination
38. Dynamic SQL
39. XML and JSON

- 40. Error Handling
 - 41. Backup and Restore
 - 42. SQL Server Agent
 - 43. Security
 - 44. Permissions and Roles
 - 45. Replication
 - 46. Always On Availability Groups
 - 47. Partitioning
 - 48. SQL Profiler and Monitoring
 - 49. Important Interview Questions
 - 50. Scenario-Based Questions
 - 51. Real-Time Optimization Examples
 - 52. Best Practices
 - 53. Frequently Asked Queries
 - 54. Advanced Topics
 - 55. HR + Project Related Questions
-

1. Introduction to MSSQL

Microsoft SQL Server is a Relational Database Management System (RDBMS) developed by Microsoft.

Features

- High performance
- Security
- Scalability
- Backup and recovery
- Transaction support
- Integration services
- Reporting services

Components

- Database Engine
 - SQL Server Agent
 - SSIS
 - SSRS
 - SSAS
-

2. SQL Server Architecture

Logical Architecture

- Database
- Tables
- Views
- Procedures
- Functions

Physical Architecture

- Data Files (.mdf)
- Log Files (.ldf)
- Secondary Files (.ndf)

Memory Architecture

- Buffer Cache
- Procedure Cache

Important Services

- SQL Server Service
- SQL Agent Service
- Browser Service

3. SQL Server Editions

Edition	Purpose
Express	Small applications
Standard	Medium scale applications
Enterprise	Large enterprise applications
Developer	Development/testing

4. Databases and Tables

Create Database

```
CREATE DATABASE CompanyDB;
```

Use Database

```
USE CompanyDB;
```

Create Table

```
CREATE TABLE Employees (  
    EmployeeID INT PRIMARY KEY,  
    Name VARCHAR(100),  
    Salary DECIMAL(10,2)  
);
```

5. Data Types

Numeric Types

- INT
- BIGINT
- SMALLINT
- DECIMAL
- FLOAT

String Types

- CHAR
- VARCHAR
- NVARCHAR
- TEXT

Date Types

- DATE
- DATETIME

- DATETIME2

Binary Types

- BINARY
 - VARBINARY
-

6. Constraints

Types of Constraints

- PRIMARY KEY
- FOREIGN KEY
- UNIQUE
- CHECK
- DEFAULT
- NOT NULL

Example

```
CREATE TABLE Department (  
    DeptID INT PRIMARY KEY,  
    DeptName VARCHAR(50) UNIQUE  
);
```

7. CRUD Operations

Insert

```
INSERT INTO Employees VALUES (1, 'John', 50000);
```

Select

```
SELECT * FROM Employees;
```

Update

```
UPDATE Employees  
SET Salary = 60000  
WHERE EmployeeID = 1;
```

Delete

```
DELETE FROM Employees  
WHERE EmployeeID = 1;
```

8. SQL Clauses

WHERE

```
SELECT * FROM Employees  
WHERE Salary > 50000;
```

ORDER BY

```
SELECT * FROM Employees  
ORDER BY Salary DESC;
```

DISTINCT

```
SELECT DISTINCT DeptID FROM Employees;
```

TOP

```
SELECT TOP 5 * FROM Employees;
```

9. Joins

INNER JOIN

```
SELECT e.Name, d.DeptName
FROM Employees e
INNER JOIN Department d
ON e.DeptID = d.DeptID;
```

LEFT JOIN

```
SELECT *
FROM Employees e
LEFT JOIN Department d
ON e.DeptID = d.DeptID;
```

RIGHT JOIN

```
SELECT *
FROM Employees e
RIGHT JOIN Department d
ON e.DeptID = d.DeptID;
```

FULL JOIN

```
SELECT *
FROM Employees e
FULL OUTER JOIN Department d
ON e.DeptID = d.DeptID;
```

SELF JOIN

```
SELECT A.Name, B.Name AS Manager
FROM Employees A
JOIN Employees B
ON A.ManagerID = B.EmployeeID;
```

CROSS JOIN

```
SELECT * FROM A CROSS JOIN B;
```

10. Set Operators

UNION

```
SELECT Name FROM Employees  
UNION  
SELECT Name FROM Managers;
```

UNION ALL

```
SELECT Name FROM Employees  
UNION ALL  
SELECT Name FROM Managers;
```

INTERSECT

```
SELECT Name FROM Employees  
INTERSECT  
SELECT Name FROM Managers;
```

EXCEPT

```
SELECT Name FROM Employees  
EXCEPT  
SELECT Name FROM Managers;
```


11. Functions

Scalar Functions

- Returns single value

Table Valued Functions

- Returns table

Built-in Functions

- LEN()
 - GETDATE()
 - COUNT()
-

12. String Functions

```
SELECT LEN('MSSQL');  
SELECT UPPER('sql');  
SELECT LOWER('SQL');  
SELECT SUBSTRING('DATABASE', 1, 4);  
SELECT REPLACE('SQL SERVER', 'SERVER', 'DB');
```

13. Date Functions

```
SELECT GETDATE();  
SELECT DATEADD(DAY, 5, GETDATE());  
SELECT DATEDIFF(DAY, '2025-01-01', GETDATE());  
SELECT YEAR(GETDATE());
```

14. Aggregate Functions

```
SELECT COUNT(*) FROM Employees;  
SELECT SUM(Salary) FROM Employees;  
SELECT AVG(Salary) FROM Employees;
```

```
SELECT MIN(Salary) FROM Employees;  
SELECT MAX(Salary) FROM Employees;
```

15. Group By and Having

```
SELECT DeptID, COUNT(*)  
FROM Employees  
GROUP BY DeptID  
HAVING COUNT(*) > 5;
```

16. Subqueries

Single Row Subquery

```
SELECT * FROM Employees  
WHERE Salary > (  
    SELECT AVG(Salary) FROM Employees  
);
```

Correlated Subquery

```
SELECT Name  
FROM Employees e  
WHERE Salary > (  
    SELECT AVG(Salary)  
    FROM Employees  
    WHERE DeptID = e.DeptID  
);
```

17. Common Table Expressions (CTE)

```
WITH EmployeeCTE AS (  
    SELECT EmployeeID, Name  
    FROM Employees  
);
```

```
)  
SELECT * FROM EmployeeCTE;
```

Recursive CTE

```
WITH Numbers AS (  
    SELECT 1 AS Num  
    UNION ALL  
    SELECT Num + 1  
    FROM Numbers  
    WHERE Num < 10  
)  
SELECT * FROM Numbers;
```

18. Temporary Tables

Local Temp Table

```
CREATE TABLE #TempEmployees (  
    ID INT,  
    Name VARCHAR(100)  
);
```

Global Temp Table

```
CREATE TABLE ##GlobalTemp (  
    ID INT  
);
```

19. Table Variables

```
DECLARE @Employees TABLE (  
    ID INT,  
    Name VARCHAR(50)  
);
```

Temp Table vs Table Variable

Temp Table	Table Variable
Stored in tempdb	Stored in memory/tempdb
Better for large data	Better for small data
Supports indexes	Limited indexing

20. Views

Create View

```
CREATE VIEW vwEmployeeDetails AS  
SELECT Name, Salary  
FROM Employees;
```

Advantages

- Security
- Reusable queries
- Simplifies complex joins

21. Indexes

Clustered Index

- Physically sorts data
- One per table

Non-Clustered Index

- Separate structure
- Multiple allowed

Create Index

```
CREATE INDEX IX_Employee_Name  
ON Employees(Name);
```

Composite Index

```
CREATE INDEX IX_Employee_Dept_Salary  
ON Employees(DepartmentID, Salary);
```

Included Columns

```
CREATE INDEX IX_Test  
ON Employees(Name)  
INCLUDE (Salary);
```

Index Fragmentation

- Internal fragmentation
- External fragmentation

Rebuild vs Reorganize

Rebuild	Reorganize
Faster	Slower
Uses more resources	Uses fewer resources
Offline mostly	Online

22. Stored Procedures

Create Procedure

```
CREATE PROCEDURE GetEmployees  
AS  
BEGIN
```

```
SELECT * FROM Employees;  
END;
```

Execute

```
EXEC GetEmployees;
```

Procedure with Parameters

```
CREATE PROCEDURE GetEmployeeByID  
    @ID INT  
AS  
BEGIN  
    SELECT * FROM Employees  
    WHERE EmployeeID = @ID;  
END;
```

Advantages

- Performance
- Security
- Reusability

23. User Defined Functions (UDF)

Scalar Function

```
CREATE FUNCTION dbo.GetBonus(@Salary DECIMAL(10,2))  
RETURNS DECIMAL(10,2)  
AS  
BEGIN  
    RETURN @Salary * 0.10;  
END;
```

Table Valued Function

```
CREATE FUNCTION dbo.GetEmployees()  
RETURNS TABLE  
AS
```

```
RETURN (  
    SELECT * FROM Employees  
);
```

24. Triggers

AFTER Trigger

```
CREATE TRIGGER trgAfterInsert  
ON Employees  
AFTER INSERT  
AS  
BEGIN  
    PRINT 'Record Inserted';  
END;
```

INSTEAD OF Trigger

```
CREATE TRIGGER trgInsteadDelete  
ON Employees  
INSTEAD OF DELETE  
AS  
BEGIN  
    PRINT 'Delete Not Allowed';  
END;
```

25. Transactions

```
BEGIN TRANSACTION;  
  
UPDATE Accounts  
SET Balance = Balance - 1000  
WHERE ID = 1;  
  
UPDATE Accounts  
SET Balance = Balance + 1000  
WHERE ID = 2;
```

```
COMMIT;
```

Rollback

```
ROLLBACK;
```

26. ACID Properties

Property	Meaning
Atomicity	All or nothing
Consistency	Valid state
Isolation	Independent transactions
Durability	Permanent changes

27. Locks and Blocking

Lock Types

- Shared Lock
- Exclusive Lock
- Update Lock
- Intent Lock

Blocking

Occurs when one transaction prevents another.

28. Isolation Levels

Isolation Level	Dirty Read
Read Uncommitted	Yes
Read Committed	No

Isolation Level	Dirty Read
Repeatable Read	No
Serializable	No
Snapshot	No

Example

```
SET TRANSACTION ISOLATION LEVEL SERIALIZABLE;
```

29. Deadlocks

Causes

- Resource contention
- Improper transaction order

Prevention

- Keep transactions short
- Access objects in same order
- Use indexes

30. Execution Plans

Types

- Estimated Execution Plan
- Actual Execution Plan

Common Operators

- Table Scan
- Index Seek
- Index Scan
- Nested Loop
- Hash Match

31. Query Optimization

Tips

- Avoid SELECT *
- Use indexes properly
- Avoid functions in WHERE clause
- Use EXISTS instead of IN for large data
- Use proper joins

Bad Query

```
SELECT * FROM Employees  
WHERE YEAR(HireDate) = 2024;
```

Optimized Query

```
SELECT * FROM Employees  
WHERE HireDate >= '2024-01-01'  
AND HireDate < '2025-01-01';
```

32. Normalization

1NF

- No repeating groups

2NF

- No partial dependency

3NF

- No transitive dependency

BCNF

- Advanced normalization
-

33. Denormalization

Used for improving read performance.

Advantages:

- Faster reads

Disadvantages:

- Data redundancy
-

34. Keys in SQL Server

Key	Purpose
Primary Key	Unique row
Foreign Key	Relationship
Candidate Key	Possible primary key
Composite Key	Multiple columns
Alternate Key	Candidate not chosen

35. Identity Column

```
CREATE TABLE Test (  
    ID INT IDENTITY(1,1),  
    Name VARCHAR(50)  
);
```

36. Sequences

```
CREATE SEQUENCE Seq_Test  
START WITH 1  
INCREMENT BY 1;
```

37. Pagination

```
SELECT *  
FROM Employees  
ORDER BY EmployeeID  
OFFSET 10 ROWS  
FETCH NEXT 10 ROWS ONLY;
```

38. Dynamic SQL

```
DECLARE @SQL NVARCHAR(MAX);  
SET @SQL = 'SELECT * FROM Employees';  
EXEC sp_executesql @SQL;
```

39. XML and JSON

FOR JSON

```
SELECT * FROM Employees  
FOR JSON AUTO;
```

OPENJSON

```
SELECT *  
FROM OPENJSON('{"Name": "John"}');
```

40. Error Handling

```
BEGIN TRY  
    SELECT 1/0;  
END TRY  
BEGIN CATCH
```

```
PRINT ERROR_MESSAGE();  
END CATCH;
```

41. Backup and Restore

Backup

```
BACKUP DATABASE CompanyDB  
TO DISK = 'C:\Backup\CompanyDB.bak';
```

Restore

```
RESTORE DATABASE CompanyDB  
FROM DISK = 'C:\Backup\CompanyDB.bak';
```

42. SQL Server Agent

Used for:

- Scheduling jobs
 - Backup jobs
 - Maintenance tasks
 - Alerts
-

43. Security

Authentication Types

- Windows Authentication
- SQL Server Authentication

Encryption

- TDE
 - Column encryption
-

44. Permissions and Roles

Grant Permission

```
GRANT SELECT ON Employees TO User1;
```

Revoke

```
REVOKE SELECT ON Employees FROM User1;
```

45. Replication

Types

- Snapshot Replication
 - Transactional Replication
 - Merge Replication
-

46. Always On Availability Groups

High availability feature in SQL Server.

Features:

- Automatic failover
 - Readable secondary replicas
 - Disaster recovery
-

47. Partitioning

Used for managing large tables.

Advantages:

- Faster query performance
- Easier maintenance

48. SQL Profiler and Monitoring

Tools

- SQL Profiler
- Extended Events
- Activity Monitor
- DMVs

Important DMV

```
SELECT * FROM sys.dm_exec_requests;
```

49. Important Interview Questions

SQL Basics

1. Difference between DELETE, TRUNCATE, and DROP?
2. Difference between WHERE and HAVING?
3. Difference between UNION and UNION ALL?
4. What is normalization?
5. What is denormalization?

Joins

1. Explain all joins with examples.
2. Difference between INNER JOIN and LEFT JOIN?
3. What is SELF JOIN?

Indexes

1. Clustered vs Non-clustered index?
2. What is index fragmentation?
3. What is covering index?
4. What is composite index?

Procedures and Functions

1. Difference between procedure and function?
2. Can function modify data?
3. Types of functions?

Transactions

1. What are ACID properties?
2. What is deadlock?
3. What is blocking?
4. What are isolation levels?

Performance

1. How to optimize slow query?
 2. What causes table scan?
 3. What is execution plan?
 4. Difference between index seek and index scan?
-

50. Scenario-Based Questions

Scenario 1

A query is slow on production. What steps will you take?

Answer

1. Check execution plan
 2. Verify indexes
 3. Check statistics
 4. Avoid SELECT *
 5. Check blocking/deadlocks
 6. Update statistics
-

Scenario 2

How will you delete duplicate records?

```
WITH CTE AS (  
    SELECT *,  
           ROW_NUMBER() OVER(PARTITION BY Name ORDER BY ID) AS RN  
    FROM Employees  
)  
DELETE FROM CTE WHERE RN > 1;
```

Scenario 3

Find second highest salary.

```
SELECT MAX(Salary)
FROM Employees
WHERE Salary < (
    SELECT MAX(Salary)
    FROM Employees
);
```

Scenario 4

Find nth highest salary.

```
SELECT Salary
FROM (
    SELECT Salary,
           DENSE_RANK() OVER(ORDER BY Salary DESC) AS RN
    FROM Employees
) A
WHERE RN = 3;
```

51. Real-Time Optimization Examples

Example 1

Problem

Query taking 20 seconds.

Solution

- Added non-clustered index
- Reduced columns
- Removed scalar function from WHERE clause

Performance improved to 1 second.

Example 2

Problem

Deadlocks in order processing.

Solution

- Reduced transaction scope
 - Accessed tables in same order
 - Added proper indexes
-

52. Best Practices

- Use naming conventions
 - Avoid SELECT *
 - Use stored procedures
 - Maintain indexes
 - Backup databases regularly
 - Use transactions carefully
 - Use proper data types
 - Avoid cursors when possible
-

53. Frequently Asked Queries

Find Duplicate Records

```
SELECT Name, COUNT(*)  
FROM Employees  
GROUP BY Name  
HAVING COUNT(*) > 1;
```

Delete Duplicate Records

```
WITH CTE AS (  
SELECT *,  
ROW_NUMBER() OVER(PARTITION BY Name ORDER BY ID) RN  
FROM Employees
```

```
)  
DELETE FROM CTE WHERE RN > 1;
```

Fetch Odd Rows

```
SELECT *  
FROM (  
    SELECT *, ROW_NUMBER() OVER(ORDER BY ID) RN  
    FROM Employees  
) A  
WHERE RN % 2 = 1;
```

Fetch Even Rows

```
SELECT *  
FROM (  
    SELECT *, ROW_NUMBER() OVER(ORDER BY ID) RN  
    FROM Employees  
) A  
WHERE RN % 2 = 0;
```

54. Advanced Topics

Window Functions

```
SELECT Name,  
       Salary,  
       ROW_NUMBER() OVER(ORDER BY Salary DESC) AS RN,  
       RANK() OVER(ORDER BY Salary DESC) AS Rnk,  
       DENSE_RANK() OVER(ORDER BY Salary DESC) AS DRnk  
FROM Employees;
```

LEAD and LAG

```
SELECT Salary,  
       LAG(Salary) OVER(ORDER BY Salary) AS PrevSalary,  
       LEAD(Salary) OVER(ORDER BY Salary) AS NextSalary  
FROM Employees;
```

MERGE Statement

```
MERGE TargetTable AS T
USING SourceTable AS S
ON T.ID = S.ID
WHEN MATCHED THEN
    UPDATE SET T.Name = S.Name
WHEN NOT MATCHED THEN
    INSERT(ID, Name)
    VALUES(S.ID, S.Name);
```

Pivot

```
SELECT *
FROM (
    SELECT DeptID, Salary
    FROM Employees
) A
PIVOT (
    SUM(Salary)
    FOR DeptID IN ([1],[2],[3])
) P;
```

55. HR + Project Related Questions

Common HR Questions

1. Tell me about yourself.
2. Explain your current project.
3. What was your role?
4. What challenges did you face?
5. Explain one optimization you did.
6. Why do you want to change your company?

Real Project Explanation Example

Project

Banking Management System

Responsibilities

- Writing stored procedures
- Optimizing SQL queries
- Creating indexes
- Handling deadlocks
- Database monitoring
- Writing reports
- Supporting production issues

Tools Used

- SQL Server
- SSMS
- SQL Profiler
- Azure DevOps

MSSQL Quick Revision Notes

Important Differences

Topic	Difference
DELETE vs TRUNCATE	DELETE logs row-by-row, TRUNCATE removes all pages
Clustered vs Non-Clustered	Clustered sorts data physically
Function vs Procedure	Function returns value
UNION vs UNION ALL	UNION removes duplicates
WHERE vs HAVING	WHERE before grouping

Top 25 Most Asked Queries

Top 5 Records

```
SELECT TOP 5 * FROM Employees;
```

Highest Salary

```
SELECT MAX(Salary) FROM Employees;
```

Count Employees by Department

```
SELECT DeptID, COUNT(*)  
FROM Employees  
GROUP BY DeptID;
```

Employees Joined Last 30 Days

```
SELECT *  
FROM Employees  
WHERE HireDate >= DATEADD(DAY, -30, GETDATE());
```

Employees Without Department

```
SELECT *  
FROM Employees  
WHERE DeptID IS NULL;
```

Important System Databases

Database	Purpose
master	System information
model	Template database
msdb	SQL agent data
tempdb	Temporary objects

SQL Server Interview Tips

1. Explain concepts with examples.

2. Focus on real-time scenarios.
 3. Learn execution plans.
 4. Practice optimization.
 5. Understand indexing deeply.
 6. Be confident with joins and window functions.
 7. Know difference-based questions clearly.
-

Final Checklist Before Interview

Must Know Topics

- Joins
 - Indexes
 - Procedures
 - Functions
 - Transactions
 - Isolation levels
 - Deadlocks
 - Execution plans
 - Query optimization
 - Window functions
 - CTEs
 - Temp tables
 - Normalization
 - Backup and restore
-

Conclusion

This document covers almost all important MSSQL topics required for:

- 2 to 4 years experience interviews
- Backend developer interviews
- SQL developer interviews
- Database administrator basics
- Production support SQL rounds

Practice all queries manually in SSMS for better understanding and interview confidence.